Rejoignez la séance

Scannez le QR code ou entrez le code sur votre téléphone.

COURS

Sécurité

Qui es-tu ? Qu'as-tu le droit de faire ?

Speaker notes

Même fil rouge que le cours middleware : FraisDirect, l'épicerie en ligne. Cette fois, elle a besoin de savoir qui se connecte et de limiter ce que chacun peut faire.

FraisDirect a besoin de comptes

- **Alice**, cliente : elle passe et suit ses commandes
- **Bruno**, préparateur : il voit les commandes à préparer
- **Chloé**, administratrice : elle gère le catalogue

Personne d'autre ne doit voir les commandes d'Alice.

Speaker notes

Ces trois personnes reviennent dans tout le cours. Demander aux étudiants ce que chacun doit pouvoir faire ou non.

Les trois A

1. **Authentification** : qui es-tu ?
2. **Autorisation** : as-tu le droit de faire ça ?
3. **Accounting** (traçabilité) : qu'as-tu fait ?

Speaker notes

On parle de AAA. L'ordre compte : on ne peut pas décider des droits de quelqu'un qu'on n'a pas identifié, ni tracer ses actions.

Au programme

1. Prouver qui on est : **mots de passe**
2. Protéger une API : **Spring Security**
3. Ne plus renvoyer son mot de passe : **tokens et JWT**
4. Se connecter ailleurs : **SSO, OAuth, OpenID Connect**
5. Les droits : **RBAC et ABAC**
6. Garder une trace : **accounting**

ACTE 1

Qui es-tu ?

Prouver son identité

Trois façons de le prouver

Ce que l'on…	Exemples
sait	un mot de passe, un code PIN
possède	un téléphone, une carte bancaire, une clé USB de sécurité
est	une empreinte digitale, un visage

Speaker notes

On appelle ça des facteurs d'authentification. On y revient avec l'authentification forte, qui en combine plusieurs.

QUESTION

Un code reçu par SMS prouve quelque chose que l'utilisateur…

A. sait

B. possède

C. est

Speaker notes

Réponse : B. Le code change à chaque fois : ce qui compte, c'est d'avoir le téléphone en main. Le code lui-même ne se retient pas.

Le mot de passe : comment l'envoyer ?

- **Basic Auth** : dans un header, à chaque requête
- **Digest Auth** : une variante de Basic, à éviter
- **Formulaire** : une requête dédiée à la connexion

Speaker notes

On détaille Basic Auth : c'est le plus simple et c'est celui qu'on utilisera avec Spring Security.

Basic Auth, étape par étape



Speaker notes

Le 401 avec le header WWW-Authenticate indique au navigateur qu'il doit demander un identifiant et un mot de passe : c'est la petite fenêtre grise du navigateur. Ensuite, le navigateur renvoie le header Authorization à chaque requête.

Construire le header

1. Identifiant et mot de passe : `alice:tomate42`
2. Encodés en Base64 : `YWxpY2U6dG9tYXRlNDI=`
3. Dans le header : `Authorization: Basic YWxpY2U6dG9tYXRlNDI=`

Speaker notes

À faire tester : echo -n 'alice:tomate42' | base64 dans un terminal, puis le décodage avec base64 -d.

QUESTION

Avec Basic Auth, le mot de passe envoyé est-il chiffré ?

Speaker notes

Réponse : non. Base64 est un encodage, pas un chiffrement : n'importe qui le décode, sans clé.

Base64 n'est pas du chiffrement

```
$ echo 'YWxpY2U6dG9tYXRlNDI=' | base64 -d
alice:tomate42
```

Basic Auth ne s'utilise **qu'en HTTPS** : c'est HTTPS qui chiffre la requête.

Speaker notes

Base64 sert à transporter n'importe quel caractère dans un header HTTP, pas à cacher quoi que ce soit.

Digest Auth : à éviter

- Le mot de passe n'est pas envoyé : le client envoie un hash calculé avec un nombre fourni par le serveur
- Pour vérifier ce hash, le serveur doit connaître le mot de passe : il ne peut pas le stocker de façon sûre
- Repose sur MD5, un algorithme cassé

Aujourd'hui : Basic Auth ou formulaire, **en HTTPS**.

Speaker notes

Digest date d'avant la généralisation de HTTPS. Il oblige le serveur à garder le mot de passe (ou un équivalent) réutilisable, ce qui contredit la slide suivante.

QUESTION

Comment FraisDirect doit-il stocker le mot de passe d'Alice en base ?

A. En clair

B. Chiffré avec une clé (AES)

C. Hashé avec SHA-256

D. Hashé avec BCrypt

Speaker notes

Réponse : D. Les slides suivantes expliquent pourquoi, étape par étape.

Stocker un mot de passe (1/3)

Jamais en clair, ni chiffré

- Les bases de données fuient : LinkedIn, Adobe, et bien d'autres
- Les gens réutilisent leurs mots de passe ailleurs
- Chiffré : celui qui vole la clé déchiffre tout

On stocke un **hash** : une empreinte qu'on ne peut pas inverser.

Speaker notes

Faire tester haveibeenpwned.com pour montrer l'ampleur des fuites. À la connexion, on hashe le mot de passe saisi et on compare les deux empreintes.

Stocker un mot de passe (2/3)

Un hash seul ne suffit pas

- Deux personnes avec le même mot de passe ont le même hash
- Des tables géantes donnent le mot de passe d'un hash connu

On ajoute un **sel** : une valeur aléatoire, différente pour chaque utilisateur, mélangée au mot de passe.

Speaker notes

Les tables précalculées (rainbow tables) contiennent les hashs des mots de passe courants. Avec un sel, il faudrait une table par utilisateur.

Stocker un mot de passe (3/3)

Un algorithme **lent**, fait pour ça : BCrypt

```
$2b$10$jpQqRmgJM0Er6Os2yTMFWu3lHfgjD7mp1xclQqTtadOgQOoSqUzcK
```

Partie	Rôle
$2b$	l'algorithme
10	le coût : 2^{10} tours
jpQqRmgJM0Er6Os2yTMFWu	le sel
3lHfgjD7mp1x…	le hash

Speaker notes

SHA-256 est conçu pour être rapide : un attaquant en teste des milliards par seconde. BCrypt est volontairement lent, et le coût s'augmente avec les années. Le sel est rangé dans le hash : pas besoin d'une colonne en plus. Hasher deux fois tomate42 donne deux résultats différents. Argon2 est l'alternative moderne.

Sans mot de passe : le certificat client

- Le client présente un certificat, comme un serveur HTTPS
- ⊕ Rien à retenir, rien à taper
- ⊖ Il faut le renouveler avant son expiration
- ⊖ Le distribuer et le révoquer est compliqué

Exemple : l'accès à l'API Kubernetes

Speaker notes

Le certificat HTTPS habituel prouve l'identité du serveur ; ici, c'est le client qui prouve la sienne. Kubernetes ne sait pas révoquer un certificat client : un certificat volé reste valable jusqu'à son expiration.

ACTE 2

Spring Security

Protéger l'API FraisDirect

Étape 1 : la dépendance

```
implementation 'org.springframework.boot:spring-boot-starter-security'
```

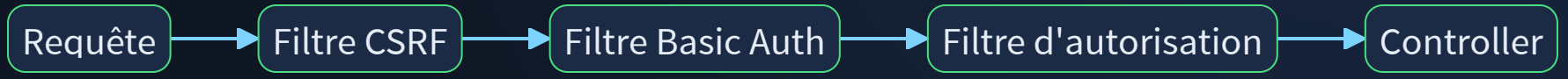
… et c'est tout : l'API est déjà protégée.

- Toutes les URL demandent une connexion
- Un utilisateur `user`, avec un mot de passe affiché dans les logs
- Un formulaire de connexion et Basic Auth

Speaker notes

Faire chercher la ligne « Using generated security password » au démarrage. Ce comportement par défaut sert à ne rien exposer par erreur : on ouvre ensuite ce qu'on veut.

Comment ça marche : des filtres



1. La requête traverse les filtres l'un après l'autre
2. Un filtre peut identifier l'utilisateur et le ranger dans le `SecurityContext`
3. Un filtre peut aussi arrêter la requête : 401 ou 403
4. Si tout passe, le controller est appelé

Speaker notes

Spring Security n'est qu'une suite de filtres (la SecurityFilterChain), placée devant les controllers. Il y en a une quinzaine en réalité ; on en montre trois.

Un filtre

```java
public class LogFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(
            HttpServletRequest request,
            HttpServletResponse response,
            FilterChain chain)
            throws ServletException, IOException {
        log.info("Requête sur {}", request.getRequestURI());
        chain.doFilter(request, response);
    }
}
```

Speaker notes

Filter est une interface de Jakarta Servlet, pas de Spring : elle existe sans Spring Security. OncePerRequestFilter (Spring) garantit un seul passage par requête. Ligne 10 : on passe la main au filtre suivant. Ne pas l'appeler, c'est arrêter la requête. Avant Spring Boot 3, les imports étaient en javax.servlet ; aujourd'hui en jakarta.servlet.

Étape 2 : configurer la chaîne

```java
@Bean
SecurityFilterChain filterChain(HttpSecurity http)
        throws Exception {
    http
        .authorizeHttpRequests(auth -> auth
            .anyRequest().authenticated())
        .httpBasic(Customizer.withDefaults());
    return http.build();
}
```

Speaker notes

Dès qu'on déclare ce bean, la configuration par défaut disparaît : on décrit tout nous-mêmes. Lignes 5-6 : toute requête doit être authentifiée. Ligne 7 : on accepte Basic Auth. Cette syntaxe avec des lambdas est celle de Spring Security 6 et 7 ; l'ancienne (.and(), antMatchers) n'existe plus.

Étape 3 : pas de session en mémoire

- Par défaut, Spring garde l'utilisateur connecté dans une session HTTP
- Cette session vit dans la mémoire de l'instance
- Avec plusieurs instances derrière un load balancer : le panier perdu du cours middleware !

```
.sessionManagement(session -> session
    .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
```

Speaker notes

STATELESS : Spring ne crée plus de session, l'utilisateur est réauthentifié à chaque requête grâce à son header. L'API peut alors tourner sur plusieurs instances.

Étape 4 : ouvrir une partie de l'API

```
1 http
2     .authorizeHttpRequests(auth -> auth
3         .requestMatchers("/produits/**").permitAll()
4         .requestMatchers("/commandes/**").authenticated()
5         .anyRequest().denyAll())
6     .httpBasic(Customizer.withDefaults());
```

Les règles sont lues **dans l'ordre** : la première qui correspond gagne.

Speaker notes

Le catalogue est public, les commandes demandent une connexion, tout le reste est refusé. Mettre anyRequest() en premier rendrait les autres règles inutiles : Spring refuse d'ailleurs de démarrer dans ce cas.

QUESTION

À vous : le catalogue est public, tout le reste demande une connexion

Le catalogue est servi sous `/produits`.

Speaker notes

Réponse attendue : .requestMatchers("/produits/**").permitAll() puis .anyRequest().authenticated(). Erreur fréquente : inverser les deux lignes.

Étape 5 : brancher nos utilisateurs

```java
@Service
public class ClientDetailsService
        implements UserDetailsService {
    // ...

    @Override
    public UserDetails loadUserByUsername(String email) {
        Client client = clientRepository.findByEmail(email)
                .orElseThrow(() ->
                        new UsernameNotFoundException(email));
        return User.withUsername(client.getEmail())
                .password(client.getMotDePasseHash())
                .roles(client.getRole())
                .build();
    }
}
```

Speaker notes

Spring Security ne connaît pas notre table de clients : UserDetailsService lui explique comment trouver un utilisateur. Spring compare ensuite lui-même le mot de passe reçu avec le hash, grâce au PasswordEncoder de la slide suivante.

Étape 6 : comparer avec BCrypt

```
@Bean
PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

À l'inscription : `passwordEncoder.encode(motDePasse)`

À la connexion : Spring appelle `matches` tout seul

Speaker notes

On ne compare jamais deux hashs BCrypt avec equals : le sel est différent à chaque encodage. C'est matches qui relit le sel dans le hash stocké.

Côté Angular : protéger les pages

```
export const authGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  const router = inject(Router);
  return auth.estConnecte()
    ? true
    : router.parseUrl('/connexion');
};
```

```
{
  path: 'commandes',
  component: CommandesComponent,
  canActivate: [authGuard]
}
```

Speaker notes

Le guard est appelé avant d'ouvrir la page : s'il renvoie une autre URL, Angular redirige. Attention : c'est du confort pour l'utilisateur, pas de la sécurité. Le code du front se modifie dans le navigateur : la vraie protection est dans l'API. Les anciens guards en classe (interface CanActivate) sont dépréciés.

Côté Angular : envoyer le header partout

```ts
export const authInterceptor: HttpInterceptorFn =
  (req, next) => {
    const token = inject(AuthService).token();
    if (!token) {
      return next(req);
    }
    return next(req.clone({
      setHeaders: { Authorization: `Bearer ${token}` }
    }));
  };
```

```ts
provideHttpClient(withInterceptors([authInterceptor]))
```

Speaker notes

L'intercepteur s'applique à toutes les requêtes HttpClient : plus besoin d'ajouter le header à la main dans chaque service. La requête est immuable : on la clone avec le header en plus. Le token Bearer arrive dans l'acte suivant.

ACTE 3

Les tokens

Ne plus envoyer son mot de passe à chaque requête

Le problème de Basic Auth

- Le mot de passe voyage dans chaque requête
- Le front doit le garder en mémoire
- Chaque requête relance BCrypt, volontairement lent

Speaker notes

BCrypt prend environ 100 ms : à chaque appel d'API, c'est beaucoup.

L'idée : échanger le mot de passe contre un token



Le mot de passe n'est envoyé qu'**une fois**.

Speaker notes

Bearer veut dire « porteur » : celui qui présente le token est authentifié. Un token volé suffit donc à se faire passer pour quelqu'un : il doit lui aussi voyager en HTTPS.

Token enregistré

1. À la connexion, l'API tire une valeur au hasard
2. Elle l'enregistre en base, avec l'utilisateur
3. À chaque requête, elle cherche le token en base

- ⊕ On peut le révoquer : il suffit de le supprimer
- ⊖ Une lecture en base à chaque requête

Speaker notes

Usages : les tokens d'API de GitHub (avec une page pour les gérer et les révoquer), l'option « rester connecté » des sites web.

Token signé

1. À la connexion, l'API écrit qui est l'utilisateur dans le token
2. Elle le signe avec un secret qu'elle seule connaît
3. À chaque requête, elle vérifie la signature : pas de base

- ⊕ Aucune base à interroger
- ⊖ Impossible à révoquer avant son expiration

Speaker notes

Pour limiter le risque : une durée de vie courte (quelques minutes) et un refresh token, enregistré et révocable, pour en obtenir un nouveau.

JWT : JSON Web Token

Le format de token signé le plus répandu : trois parties séparées par des points

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhbGljZSIsIm5hbWUiOiJBbGljZSIsInJvbGVzIjpbIkNMSUVOVCJdLCJpYXQiOjE3OTU0MzA0MDAsImV4cCI6MTc5NTQzNDAwMH0.kmSnlxdc6nSezfww2U7Qcse7mJTaiJNuLD0D9oJZLRY

1. Le header : l'algorithme de signature
2. Le payload : les informations sur l'utilisateur
3. La signature

Speaker notes

Un vrai token, signé avec le secret « secret-de-fraisdirect ». Le coller sur jwt.io pour le décoder en direct, puis saisir le secret pour vérifier la signature.

Ce qu'il contient

```
{ "alg": "HS256", "typ": "JWT" }
```

```
{
  "sub": "alice",
  "name": "Alice",
  "roles": ["CLIENT"],
  "iat": 1795430400,
  "exp": 1795434000
}
```

Speaker notes

Les deux premières parties sont du JSON encodé en Base64. sub : l'identifiant de l'utilisateur. iat : la date de création. exp : la date d'expiration, ici une heure plus tard. Ces dates sont en secondes depuis 1970.

QUESTION

Sans connaître le secret, peut-on lire le contenu d'un JWT ?

Speaker notes

Réponse : oui. Ce n'est que du Base64, comme Basic Auth.

Signé, pas chiffré

- Tout le monde peut **lire** un JWT
- Seul le détenteur du secret peut le **créer** ou le **modifier**

Jamais de mot de passe ni de donnée secrète dans un JWT.

La signature

```
HMAC-SHA256( header + "." + payload , "secret-de-fraisdirect" )
```

Pour vérifier un token, l'API :

1. sépare les trois parties
2. recalcule la signature avec son secret
3. la compare avec celle du token
4. vérifie que `exp` n'est pas dépassé

Speaker notes

Le secret ne se met jamais dans le code (le TP le faisait pour simplifier) : variable d'environnement ou gestionnaire de secrets.

QUESTION

Alice décode son JWT, remplace `CLIENT` par `ADMIN` et le renvoie à l'API. Que se passe-t-il ?

A. L'API l'accepte : Alice devient administratrice

B. L'API le rejette : la signature ne correspond plus

C. L'API lui redemande son mot de passe

Speaker notes

Réponse : B. Le payload a changé, la signature recalculée par l'API est différente de celle du token. Pour en produire une valide, il faudrait le secret.

ACTE 4

Se connecter ailleurs

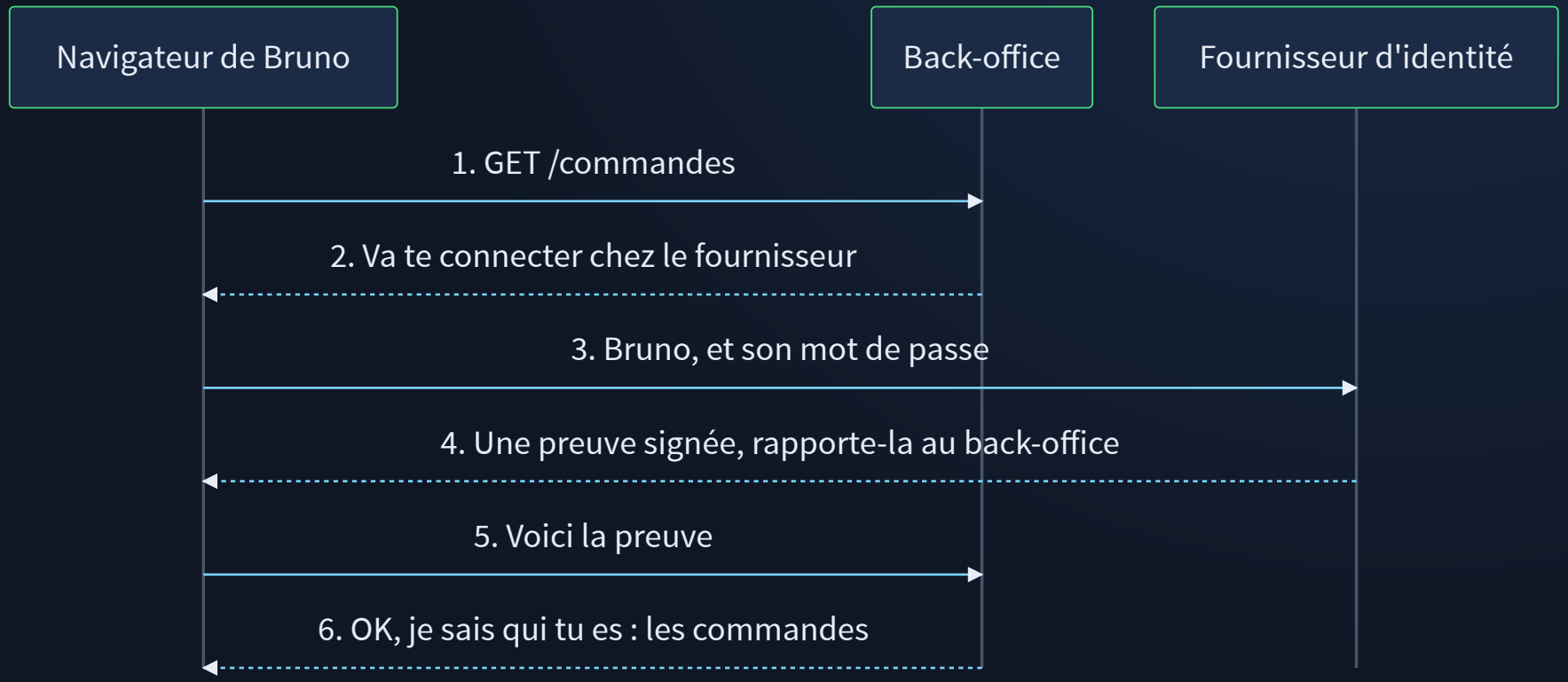
SSO, authentification forte, OAuth, OpenID Connect

Le problème des employés

- Bruno utilise le back-office, l'outil de l'entrepôt, le planning…
- Un compte et un mot de passe par application
- Et le jour où il quitte FraisDirect, il faut les fermer un par un

Une seule connexion pour toutes les applications : le **Single Sign-On** (SSO).

Le SSO avec SAML



Speaker notes

Le back-office (service provider) ne voit jamais le mot de passe de Bruno. Il fait confiance au fournisseur d'identité (identity provider), dont il vérifie la signature. Pour l'outil de l'entrepôt, Bruno est déjà connecté chez le fournisseur : l'étape 3 est sautée. SAML est un standard en XML, très présent en entreprise.

Ce que le SSO apporte

- Un seul mot de passe pour toutes les applications
- On ne se connecte qu'une fois
- On peut renforcer la connexion à un seul endroit
- Branché sur l'annuaire de l'entreprise (LDAP, Active Directory)

Speaker notes

Le départ de Bruno : on désactive un seul compte, dans l'annuaire.

L'authentification forte

Combiner au moins deux facteurs de l'acte 1

- Ce qu'on **sait** : le mot de passe
- + ce qu'on **possède** : une application d'authentification, une clé de sécurité, un SMS

  - ⊕ Un mot de passe volé ne suffit plus
  - ⊖ Une connexion un peu moins simple

Speaker notes

Grâce au SSO, on ne l'ajoute qu'à un seul endroit : le fournisseur d'identité. Le SMS est le second facteur le plus faible (détournement de numéro) ; les clés de sécurité et les passkeys sont les plus solides.

Un nouveau besoin

FraisDirect veut ajouter le créneau de livraison dans l'agenda Google d'Alice.

Mauvaise idée : demander à Alice son mot de passe Google.

Bonne idée : demander à Google une autorisation **limitée** à l'agenda : **OAuth 2**.

Speaker notes

Avec le mot de passe, FraisDirect pourrait tout faire : lire les mails, changer le mot de passe. OAuth donne un accès limité (le scope), révocable par Alice à tout moment.

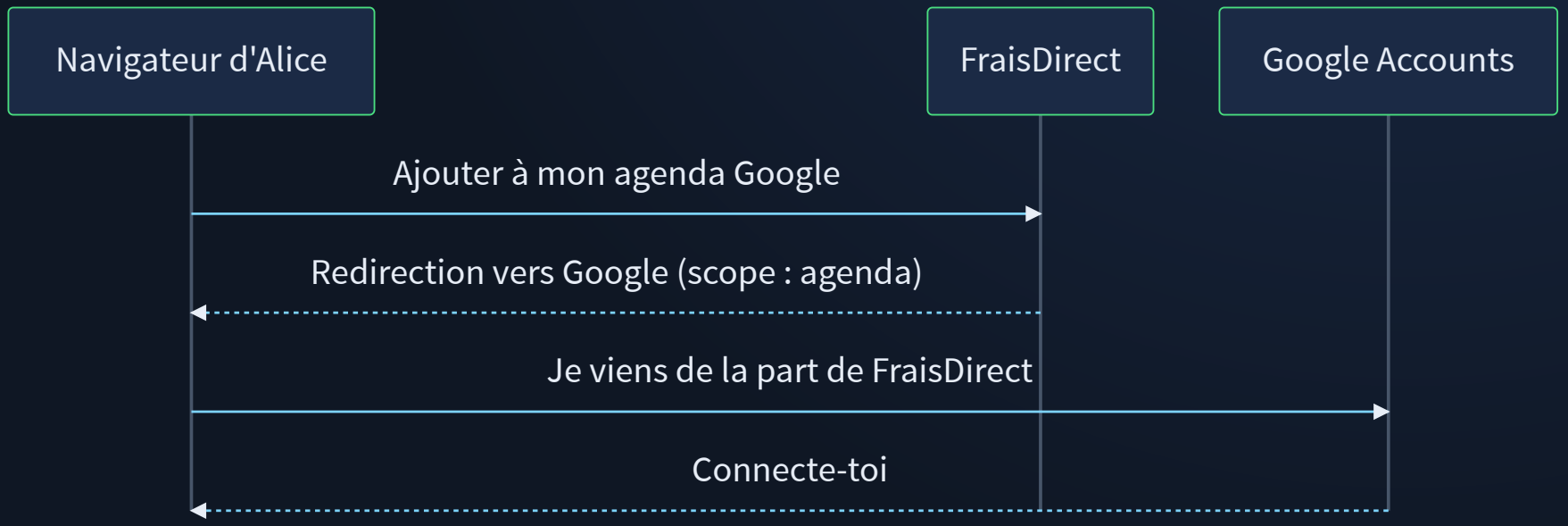
Les acteurs d'OAuth

Rôle	Dans l'exemple
Resource owner	Alice, propriétaire de son agenda
Client	FraisDirect, qui veut accéder à l'agenda
Authorization server	Google Accounts, qui délivre les tokens
Resource server	L'API Google Agenda

Speaker notes

Attention au vocabulaire : en OAuth, le « client » est l'application (FraisDirect), pas l'utilisateur.

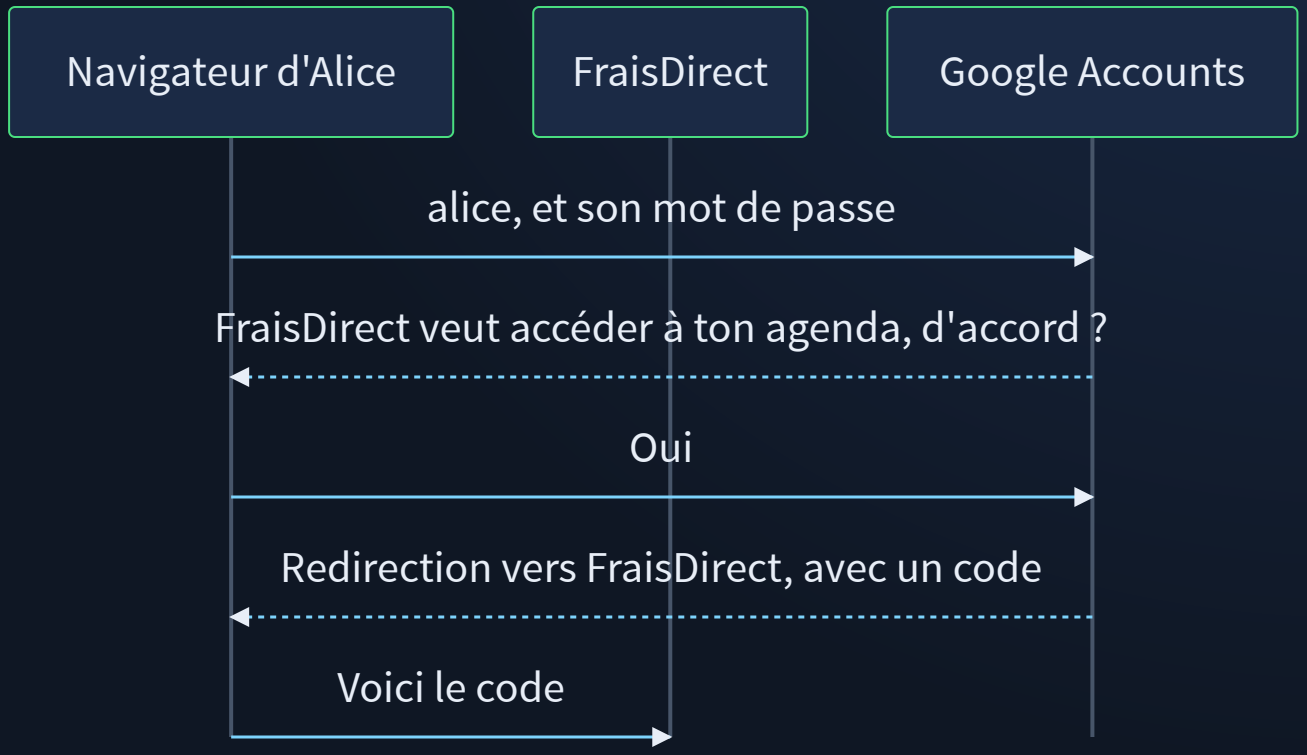
OAuth étape par étape (1/3)



Speaker notes

Alice se connecte chez Google, sur une page de Google : FraisDirect ne voit jamais son mot de passe. La redirection contient l'identifiant de FraisDirect et le scope demandé.

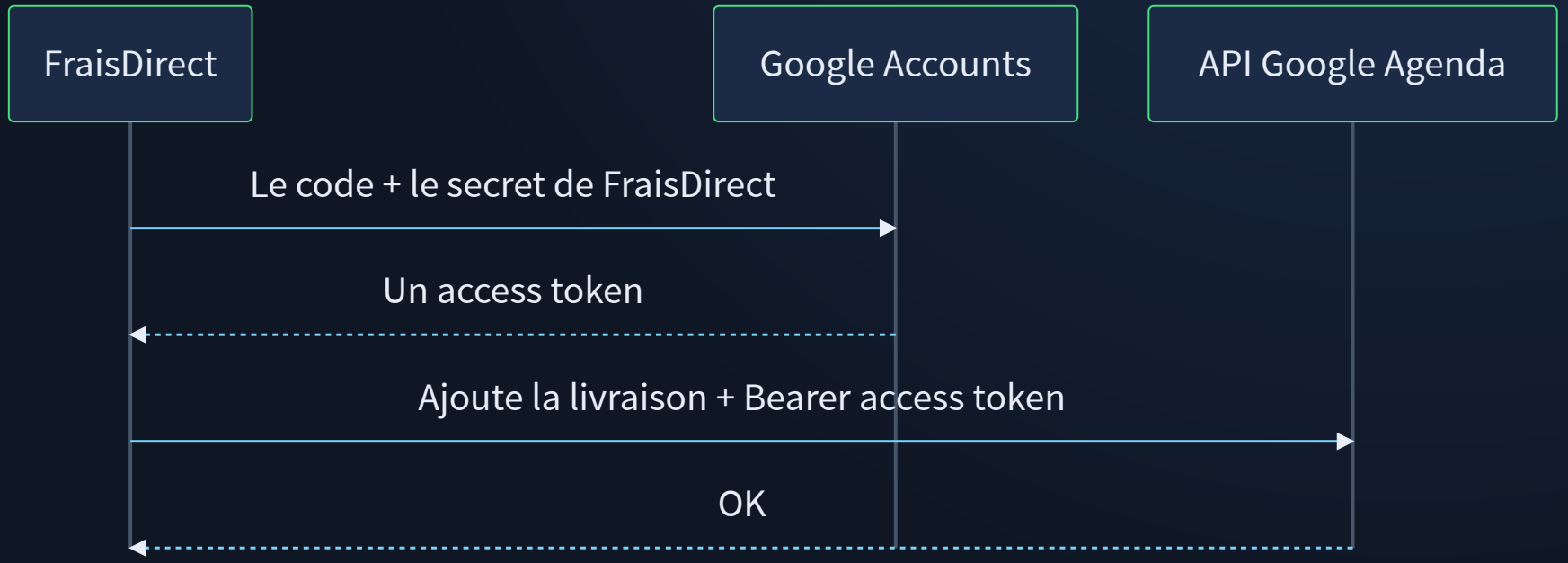
OAuth étape par étape (2/3)



Speaker notes

C'est l'écran de consentement. Le code est à usage unique et ne vit que quelques secondes : il passe par le navigateur, donc on ne veut pas qu'il serve à grand-chose.

OAuth étape par étape (3/3)



Speaker notes

Cet échange se fait de serveur à serveur, sans passer par le navigateur. C'est le flow « authorization code ». Pour une application sans serveur (Angular seul), on ajoute PKCE à la place du secret. L'ancien flow « implicit » est déconseillé.

QUESTION

Dans cet exemple, qui délivre l'access token ?

A. FraisDirect

B. Google Accounts

C. L'API Google Agenda

D. Le navigateur d'Alice

Speaker notes

Réponse : B, l'authorization server. FraisDirect est le client, l'API Agenda est le resource server.

OAuth autorise, il n'identifie pas

- L'access token dit : « ce porteur peut modifier cet agenda »
- Il ne dit pas à FraisDirect **qui** est Alice

Pour « Se connecter avec Google » : **OpenID Connect**.

OpenID Connect (OIDC)

1. Les mêmes étapes qu'OAuth, avec le scope `openid`
2. En plus de l'access token, Google renvoie un **ID token**
3. L'ID token est un JWT qui décrit l'utilisateur


```
{
  "iss": "https://accounts.google.com",
  "sub": "110248495921238986420",
  "aud": "fraisdirect",
  "email": "alice@example.com",
  "name": "Alice",
  "exp": 1795434000
}
```

Speaker notes

iss : qui a émis le token. aud : pour quelle application. sub : l'identifiant unique et stable d'Alice chez Google, à utiliser plutôt que l'email, qui peut changer.

La signature, sans secret partagé

- Google signe ses tokens avec une clé **privée** (algorithme RS256)
- Il publie la clé **publique** correspondante
- N'importe quelle API peut vérifier le token, sans pouvoir en fabriquer

Speaker notes

Avec HS256 (acte 3), le même secret sert à signer et à vérifier : on ne peut pas le partager avec toutes les applications du monde. Avec RS256, seul Google signe, tout le monde vérifie.

La découverte automatique

`https://accounts.google.com/.well-known/openid-configuration`


```
{
  "issuer": "https://accounts.google.com",
  "authorization_endpoint": "https://accounts.google.com/o/oauth2/v2/auth",
  "token_endpoint": "https://oauth2.googleapis.com/token",
  "jwks_uri": "https://www.googleapis.com/oauth2/v3/certs"
}
```

Speaker notes

Extrait. Tout fournisseur OIDC publie ce document à la même adresse : il suffit de connaître l'issuer pour trouver le reste. jwks_uri donne les clés publiques. Ouvrir l'URL en direct dans le navigateur.

Dans Spring : vérifier les tokens

```
implementation 'org.springframework.boot:spring-boot-starter-oauth2-resource-server'
```

```
spring.security.oauth2.resourceserver.jwt.issuer-uri=https://accounts.google.com
```

```
http.oauth2ResourceServer(oauth2 -> oauth2
        .jwt(Customizer.withDefaults()));
```

1. Au démarrage, Spring lit la découverte de l'issuer
2. Il récupère les clés publiques
3. À chaque requête, il vérifie la signature et l'expiration du token

Speaker notes

La ligne Java se place dans la SecurityFilterChain, à la place de httpBasic. Pour avoir son propre fournisseur d'identité (comptes clients de FraisDirect), on installe Keycloak : seul l'issuer change.

ACTE 5

As-tu le droit ?

L'autorisation

QUESTION

Bruno est bien connecté, mais il n'a pas le droit de modifier le catalogue. Quel code HTTP l'API doit-elle renvoyer ?

A. 401 Unauthorized

B. 403 Forbidden

Speaker notes

Réponse : B. Piège classique : malgré son nom, 401 veut dire « je ne sais pas qui tu es » (authentification), 403 « je sais qui tu es, mais c'est non » (autorisation).

401 ou 403 ?

Code	Signifie	Le client doit…
401	Je ne sais pas qui tu es	se connecter
403	Je sais qui tu es, et c'est non	abandonner

RBAC : des droits par rôle

Rôle	Catalogue	Commandes
CLIENT (Alice)	lire	passer, suivre
PREPARATEUR (Bruno)	lire	voir, préparer
ADMIN (Chloé)	lire, modifier	tout

Speaker notes

Role Based Access Control : on donne des droits à des rôles, puis des rôles aux personnes. Exemple d'API qui fonctionne ainsi : Kubernetes.

Les rôles dans Spring

1. `.roles("ADMIN")` dans le `UserDetailsService`
2. Spring en fait l'autorité `ROLE_ADMIN`
3. `hasRole("ADMIN")` cherche `ROLE_ADMIN`
4. `hasAuthority("ADMIN")` cherche exactement `ADMIN`

Speaker notes

Source d'erreurs fréquente : mélanger hasRole et hasAuthority. Choisir une convention et s'y tenir.

Des règles par URL

```
1 http
2     .authorizeHttpRequests(auth -> auth
3         .requestMatchers(HttpMethod.GET, "/produits/**")
4             .permitAll()
5         .requestMatchers("/produits/**")
6             .hasRole("ADMIN")
7         .requestMatchers("/preparation/**")
8             .hasAnyRole("PREPARATEUR", "ADMIN")
9         .anyRequest().authenticated())
```

Speaker notes

Lignes 3-4 : lire le catalogue est public. Lignes 5-6 : le modifier (POST, PUT, DELETE) est réservé aux administrateurs. L'ordre compte toujours.

Piège : mon POST renvoie 403

- Spring active la protection **CSRF** par défaut
- Elle protège les sessions par cookie
- Notre API est stateless, avec un header : pas de cookie à protéger

```
.csrf(csrf -> csrf.disable())
```

Speaker notes

CSRF : un site malveillant fait envoyer une requête par le navigateur de la victime, qui joint automatiquement ses cookies. Un header Authorization n'est jamais ajouté automatiquement : l'attaque ne fonctionne pas. On ne désactive CSRF que dans ce cas-là.

Des règles sur les méthodes

```
@Configuration
@EnableMethodSecurity
public class SecurityConfig { /* ... */ }
```

```
1 @PostMapping("/produits")
2 @PreAuthorize("hasRole('ADMIN')")
3 public Produit creer(@RequestBody Produit produit) {
4     return produitService.creer(produit);
5 }
```

Speaker notes

Sans @EnableMethodSecurity, les annotations sont ignorées en silence. @Secured et @RolesAllowed existent aussi ; @PreAuthorize est la plus souple, on s'en sert juste après pour l'ABAC.

Une hiérarchie de rôles

```
@Bean
static RoleHierarchy roleHierarchy() {
    return RoleHierarchyImpl.fromHierarchy("""
            ROLE_ADMIN > ROLE_PREPARATEUR
            ROLE_PREPARATEUR > ROLE_CLIENT
            """);
}
```

Chloé a automatiquement les droits de Bruno, et Bruno ceux d'Alice.

Speaker notes

Plus besoin d'écrire hasAnyRole("PREPARATEUR", "ADMIN") : hasRole("PREPARATEUR") suffit. Disponible depuis Spring Security 6.3 ; l'ancienne version avec AccessDecisionVoter est dépréciée.

Le rôle ne suffit pas toujours

Alice et Bob sont tous les deux `CLIENT`.

Alice ne doit voir que **ses** commandes, pas celles de Bob.

La décision dépend d'un **attribut** : à qui appartient la commande.

ABAC : des droits par attribut

- Le pays de l'utilisateur : le catalogue de Netflix
- L'âge : le contrôle parental de Disney+
- La relation : les publications « amis » de Facebook
- Le propriétaire : les commandes d'Alice

Speaker notes

Attribute Based Access Control : la décision combine des attributs de l'utilisateur, de la ressource et du contexte (heure, lieu).

QUESTION

« Un préparateur ne voit que les commandes de son entrepôt. » C'est une règle…

A. RBAC : par rôle

B. ABAC : par attribut

Speaker notes

Réponse : B. Le rôle PREPARATEUR ne suffit pas : il faut comparer l'entrepôt du préparateur et celui de la commande. En pratique, les deux se combinent : le rôle donne accès aux commandes, l'attribut filtre lesquelles.

ABAC dans Spring

```java
@GetMapping("/commandes/{id}")
@PostAuthorize("returnObject.client == authentication.name")
public Commande commande(@PathVariable Long id) {
    return commandeService.trouver(id);
}
```

1. La méthode s'exécute et charge la commande
2. Spring compare son client avec l'utilisateur connecté
3. S'ils diffèrent : 403

Speaker notes

@PostAuthorize décide après l'exécution, à partir du résultat. @PreAuthorize décide avant, à partir des paramètres : @PreAuthorize("#clientId == authentication.name"). Pour des règles plus complexes, on écrit un bean et on l'appelle depuis l'annotation.

RBAC et ABAC ensemble

- Les rôles donnent les grands droits
- Les attributs les affinent

Speaker notes

Les rôles seuls ne suffisent pas pour les données personnelles ; les attributs seuls deviennent vite illisibles.

ACTE 6

Qu'as-tu fait ?

L'accounting

Garder une trace

- **Qui** : Chloé
- **Quoi** : a changé le prix des tomates de 4,20 € à 0,42 €
- **Quand** : le 24 novembre à 9 h 12

Dans des logs dédiés, ou dans une table d'historique en base.

Speaker notes

Utile pour comprendre une erreur, pour un audit, et parfois obligatoire (banque, santé, paiement). Les traces doivent être difficiles à modifier, y compris par un administrateur, et ne jamais contenir de mot de passe ni de token.

À retenir

- Authentification, autorisation, accounting : dans cet ordre
- Un mot de passe se stocke hashé avec BCrypt, et voyage en HTTPS
- Un JWT est signé, pas chiffré
- OAuth autorise, OpenID Connect authentifie
- La sécurité se fait dans l'API, jamais seulement dans le front